Lab 6: Design of a Simple General-Purpose Processor

Rohit Raju

Partner: Salah Mokahal

COE328 - Section 4

Instructor: Vadim Geurkov

TA: Nahid Sahel Gozin

Submission Date: December 1, 2025

1. Table of Contents

1. Table of Contents..... [Page 2]
2. Introduction ..... [Page 3]
3. Components ..... [Page 3]
    3.1. Latch1 ..... [Page 3]
    3.2. Latch2 ..... [Page 3]
    3.3. 4:16 Decoder ..... [Page 5]
    3.4. FSM ..... [Page 7]
4. ALU_1 for Problem Set 1 ..... [Page 9]
5. ALU_2 for Problem Set 2 ..... [Page 14]
6. ALU_3 for Problem Set 3 ..... [Page 19]
7. Conclusion ..... [Page 22]

2. Introduction

This lab involves the design and implementation of a simple GPU using VHDL. In the designed system, there are four main components interacting with each other. The Control Unit consists of a Finite State Machine (FSM) and a decoder, which control the sequencing of the operations. Storage units implemented as latches store the 8-bit input temporarily for processing stability. The ALU core executes arithmetic and logical operations on stored inputs. The results are driven out onto 7-segment displays in hexadecimal format. The design is modular, wherein every unit has a different functionality within the data path. The clock ensures that the operation of all units is well synchronized. This lab should provide hands-on experience with the integration of digital systems and the basics of processor architecture. It shows how individual components in a processing unit work together to accomplish a computational task efficiently within a structured pipeline.

3. Components: Latch1, Latch2, 4:16 Decoder, FSM

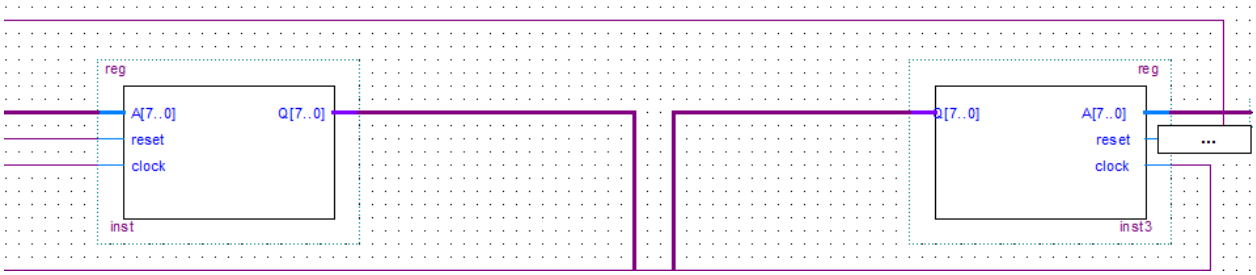
3.1 Latch1/ 3.2 Latch2

- Description: 8-bit storage registers for inputs A and B. They capture data on clock rising edge and maintain stability during ALU operations. Reset clears outputs to zero.

- Truth Table:

Reset	Clock event	A(7..0)	Next Q(7..0)	Description
0	X	XXXXXXXX	00000000	Reset active → register cleared to 0
1	↑ (rising)	00000000	00000000	Rising edge, store 00000000
1	↑	01010101	01010101	Rising edge, store 01010101
1	↑	11111111	11111111	Rising edge, store 11111111
1	0 / ↓ / −	XXXXXXXX	Q (no change)	No rising edge → Q holds its previous value

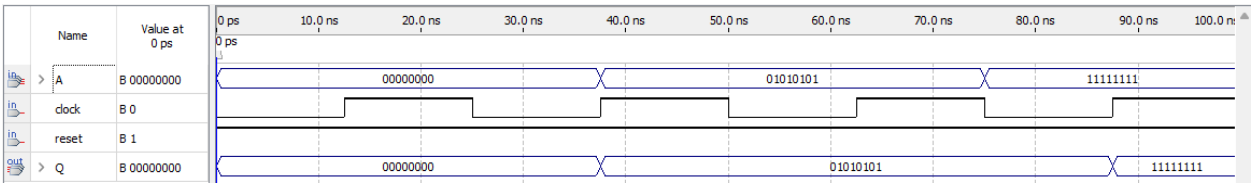
- Circuit/Block Diagram:



- VHDL Code:

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity reg is
    port(
        A     : in  std_logic_vector(7 downto 0);  -- data in
        reset : in  std_logic;                     -- active-low reset
        clock : in  std_logic;                     -- clock
        Q     : out std_logic_vector(7 downto 0)   -- data out
    );
end entity;

architecture behavior of reg is
begin
    process(reset, clock)
    begin
        if reset = '0' then
            Q <= (others => '0');          -- clear on reset
        elsif rising_edge(clock) then
            Q <= A;                        -- store A on rising edge
        end if;
    end process;
end architecture;
```

- Waveform File:



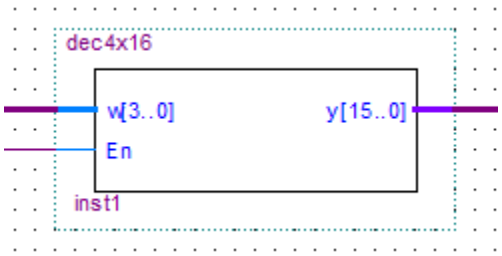
3.3 4:16 Decoder

- Description: Converts 4-bit FSM state to 16 operation signals. Uses two 3-to-8 decoders with enable logic. Ensures only one ALU operation is active per cycle by decoding current_state into microcode.

- Truth Table:

S3 S2 S1 S0	Decimal (state)	Active decoder	Output
0 0 0 0	0	Lower	OP0 = 0000000000000001
0 0 0 1	1	Lower	OP1 = 0000000000000010
0 0 1 0	2	Lower	OP2 = 0000000000000100
0 0 1 1	3	Lower	OP3 = 0000000000001000
0 1 0 0	4	Lower	OP4 = 0000000000010000
0 1 0 1	5	Lower	OP5 = 0000000000100000
0 1 1 0	6	Lower	OP6 = 0000000001000000
0 1 1 1	7	Lower	OP7 = 0000000010000000
1 0 0 0	8	Upper	OP8 = 0000000100000000
1 0 0 1	9	Upper	OP9 = 0000001000000000

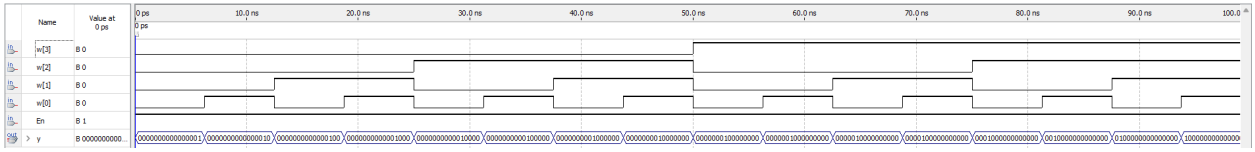
- Circuit/Block Diagram:



- VHDL Code:

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity dec4x16 is
    port (
        w   : in  std_logic_vector(3 downto 0);   -- 4-bit input
        En  : in  std_logic;                      -- enable
        y   : out std_logic_vector(15 downto 0)   -- 16 outputs
    );
end entity;

architecture behavior of dec4x16 is
begin
    process(w, En)
    begin
        if En = '1' then
            case w is
                when "0000" => y <= "0000000000000001";
                when "0001" => y <= "0000000000000010";
                when "0010" => y <= "0000000000000100";
                when "0011" => y <= "0000000000001000";
                when "0100" => y <= "0000000000010000";
                when "0101" => y <= "0000000000100000";
                when "0110" => y <= "0000000001000000";
                when "0111" => y <= "0000000010000000";
                when "1000" => y <= "0000000100000000";
                when "1001" => y <= "0000001000000000";
                when "1010" => y <= "0000010000000000";
                when "1011" => y <= "0000100000000000";
                when "1100" => y <= "0001000000000000";
                when "1101" => y <= "0010000000000000";
                when "1110" => y <= "0100000000000000";
                when "1111" => y <= "1000000000000000";
                when others => y <= (others => '0');
            end case;
        else
            y <= (others => '0'); -- disabled
        end if;
    end process;

end architecture;
```

- Waveform File:



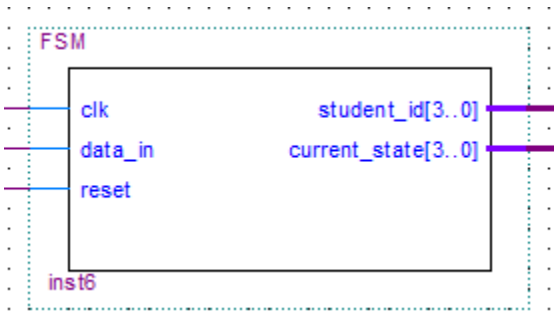
3.4 FSM

- Description: Cycles through 9 states (0-8) sequentially on clock rising edges. As a Moore machine, outputs depend only on current state. Provides student_id for conditional operations and controls operation sequencing.

- Truth Table:

Reset	Present state name	Present current_state (Q3 Q2 Q1 Q0)	Next state name	Next current_state (Q3' Q2' Q1' Q0')	Description
1	X (any)	XXXX	S0	0000	Reset → go to S0
0	S0	0000	S1	0100	S0 → S1
0	S1	0001	S2	1000	S1 → S2
0	S2	0010	S3	0001	S2 → S3
0	S3	0011	S4	0010	S3 → S4
0	S4	0100	S5	0011	S4 → S5
0	S5	0101	S6	0000	S5 → S6
0	S6	0110	S7	0101	S6 → S7
0	S7	0111	S8	0110	S7 → S8
0	S8	1000	S8	0111	S8 → S0

- Circuit/Block Diagram:

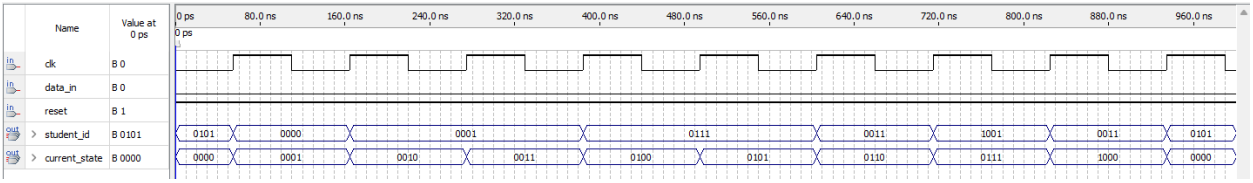


- VHDL Code:

```
library ieee;
use ieee.std_logic_1164.all;
entity FSM is
    port
    (   clk          : in  std_logic;
        data_in      : in  std_logic;
        reset        : in  std_logic;
        student_id   : out std_logic_vector(3 downto 0);
        current_state: out std_logic_vector(3 DOWNTO 0)
    );
end entity;
architecture fsm of FSM is
    -- Build an enumerated type with 9 states for
    -- the state machine ( 9 states for parsing 9 digits of
    -- student id)
    type state_type is (s0, s1, s2, s3, s4, s5, s6, s7, s8);
    -- Register to hold the current state
    signal yfsm : state_type;
begin
    process (clk, reset)
    begin
        if reset = '0' then -- active-LOW reset
            yfsm <= s0;
        elsif rising_edge(clk) then
            -- Determine the next state
            -- synchronously, based on
            -- the current state and the input
            case yfsm is
                when s0 =>      -- S0 -> S1
                    yfsm <= s1;
                when s1 =>      -- S1 -> S2
                    yfsm <= s2;
                when s2 =>      -- S2 -> S3
                    yfsm <= s3;
                when s3 =>      -- S3 -> S4
                    yfsm <= s4;
                when s4 =>      -- S4 -> S5
                    yfsm <= s5;
                when s5 =>      -- S5 -> S6
                    yfsm <= s6;
                when s6 =>      -- S6 -> S7
                    yfsm <= s7;
                when s7 =>      -- S7 -> S8
                    yfsm <= s8;
                when s8 =>      -- S8 -> S0
                    yfsm <= s0;
            end case;
        end if;
    end process;
    -- Moore
    process (yfsm, data_in) -- data_in if reqd only
    begin
        case yfsm is
            when s0 =>                      -- 5
                student_id    <= "0101";
                current_state <= "0000";    -- S0
            when s1 =>                      -- 0
                student_id    <= "0000";
                current_state <= "0001";    -- S1
            when s2 =>                      -- 1
                student_id    <= "0001";
                current_state <= "0010";    -- S2
            when s3 =>                      -- 1
                student_id    <= "0001";
                current_state <= "0011";    -- S3
            when s4 =>                      -- 7
                student_id    <= "0111";
                current_state <= "0100";    -- S4
            when s5 =>                      -- 7
```

```
    process (yfsm, data_in) -- data_in if reqd only
    begin
        case yfsm is
            when s0 =>                      -- 5
                student_id    <= "0101";
                current_state <= "0000";    -- S0
            when s1 =>                      -- 0
                student_id    <= "0000";
                current_state <= "0001";    -- S1
            when s2 =>                      -- 1
                student_id    <= "0001";
                current_state <= "0010";    -- S2
            when s3 =>                      -- 1
                student_id    <= "0001";
                current_state <= "0011";    -- S3
            when s4 =>                      -- 7
                student_id    <= "0111";
                current_state <= "0100";    -- S4
            when s5 =>                      -- 7
                student_id    <= "0111";
                current_state <= "0101";    -- S5
            when s6 =>                      -- 3
                student_id    <= "0011";
                current_state <= "0110";    -- S6
            when s7 =>                      -- 9
                student_id    <= "1001";
                current_state <= "0111";    -- S7
            when s8 =>                      -- 3
                student_id    <= "0011";
                current_state <= "1000";    -- S8
        end case;
    end process;
end fsm;
```

- Waveform File:

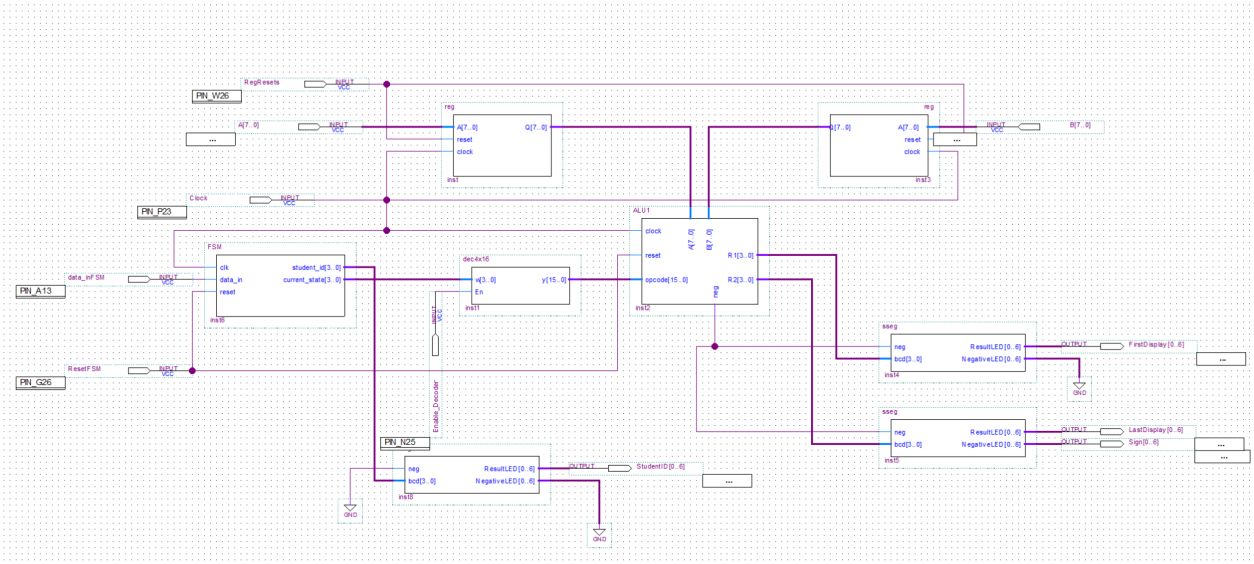


4. ALU_1 for Problem Set 1 of the Lab 6 procedure

- Description and Purpose: ALU_1 is the main execution unit that realizes nine basic arithmetic and logic operations. It has two 8-bit binary inputs (A and B) that are processed according to a 16-bit operation code from the control unit. It acts as the main execution unit in this simple GPU architecture, doing the calculation on every system clock. The ALU makes use of temporary registers to stabilize the inputs during a computation cycle. It also contains conditional logic for detecting negative results.

  Design Approach: The operation to be executed is decoded using the VHDL case statement architecture by the 16-bit opcode; afterwards, ALU performs the operation accordingly. For arithmetic operations, it uses unsigned arithmetic, while for logical operations, it performs bitwise operations. The result, an 8-bit number, is divided systematically into two 4-bit bits, R1 and R2, for hexadecimal display on two 7-segment displays.

- Block Schematic (BDF):

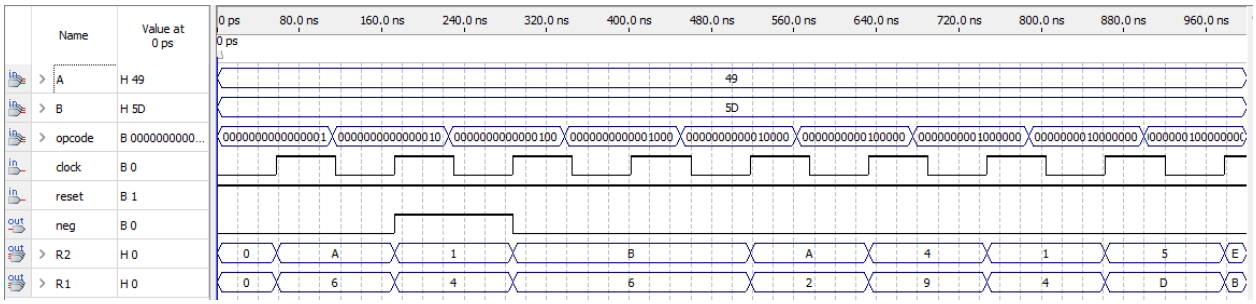


- Inputs and Outputs:
    - A and B (Input, 8-bit): Provides the operand used in all arithmetic and logic operations in ALU1.
    - Opcode (Input, 16-bit): Microcode from decoder, selects which specific ALU function to execute.
    - Clock (Input): System clock (Rising Edge), Output is updated on each rising edge.
    - Reset (Input): Active-low signal that clears the ALU result and resets the state.
    - neg (Output): Indicates whether a subtraction operation produced a negative result (B > A).
    - R1 (Output, 4-bit): Outputs the lower bits (3-0) of the 8-bit ALU result for the 7-segment display.
    - R2 (Output, 4-bit): Outputs the upper bits (7-4) of the 8-bit ALU result for the 7-segment display.

- Table of Microcode's generated by decoder for ALU_1:
- Student ID: 501177393 -> A = 73 = 01001001 (0x49), B = 93 = 01011101 (0x5D)

Function #	**Microcode (OP15 … OP0)**	**ALU1 Operation/Function**	**Results (Hex)**
1	0000 0000 0000 0001	A + B	A6
2	0000 0000 0000 0010	A - B	-14
3	0000 0000 0000 0100	$\overline{A}$	B6
4	0000 0000 0000 1000	$\overline{A \cdot B}$	B6
5	0000 0000 0001 0000	$\overline{A + B}$	A2
6	0000 0000 0010 0000	A·B	49
7	0000 0000 0100 0000	A ⊕ B	14
8	0000 0000 1000 0000	A + B	5D
9	0000 0001 0000 0000	$\overline{A \oplus B}$	EB

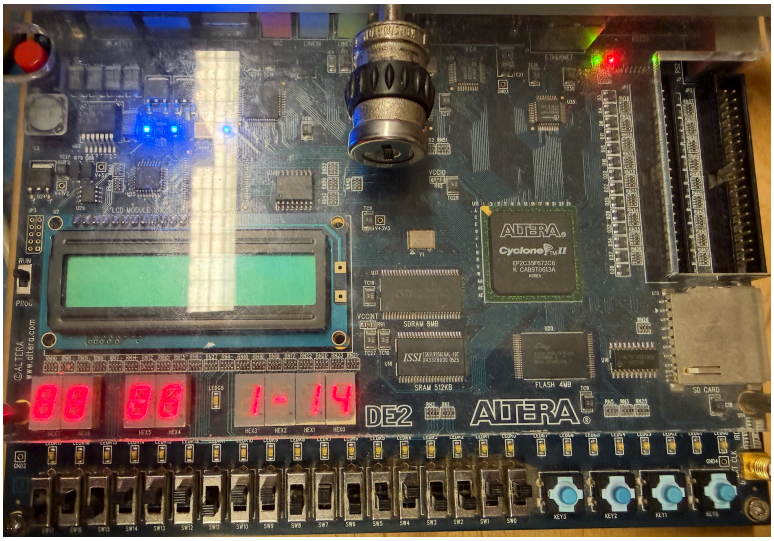
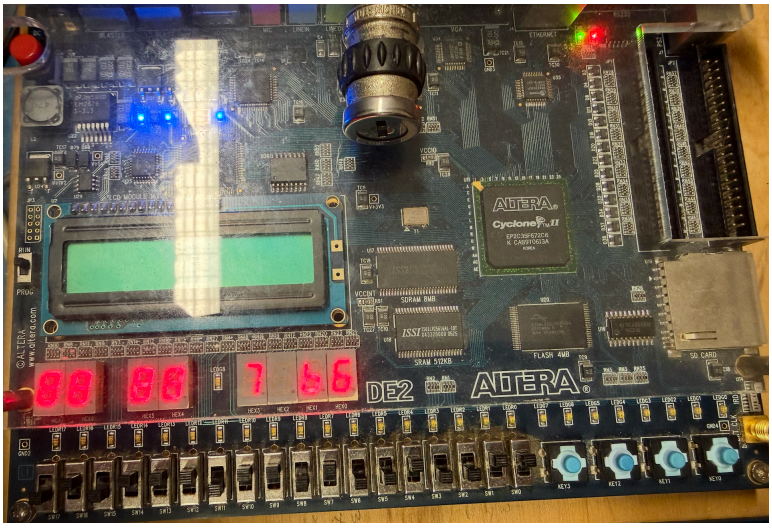
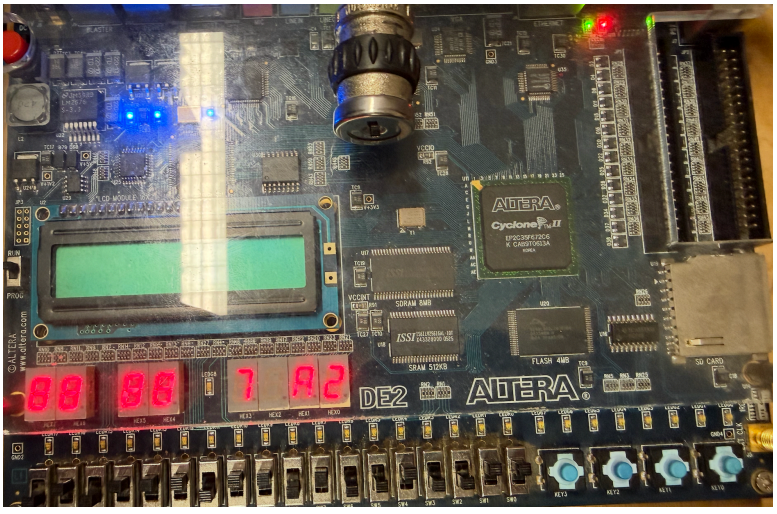
- Complete Waveform File:



- VHDL Code:

```vhdl
LIBRARY ieee;
USE ieee.std_logic_1164.all;
USE ieee.std_logic_unsigned.all;
USE ieee.numeric_std.all;

ENTITY ALU1 IS
    PORT (
        clock    : IN  std_logic;
        reset    : IN  std_logic;              -- active-low reset
        A, B     : IN  unsigned(7 downto 0);   -- 8-bit inputs
        opcode   : IN  unsigned(15 downto 0);  -- 16-bit opcode from decoder
        neg      : OUT std_logic;              -- negative flag
        R1, R2   : OUT unsigned(3 downto 0)    -- 4-bit outputs for 7-seg
    );
END ALU1;

ARCHITECTURE calculation OF ALU1 IS

    SIGNAL Reg1, Reg2, Result : unsigned(7 downto 0) := (others => '0');

BEGIN

    ---------------------------------------------------------------------
    -- Store inputs in temporary registers
    ---------------------------------------------------------------------
    Reg1 <= A;
    Reg2 <= B;

    PROCESS(clock, reset)
    BEGIN
        ---------------------------------------------------------------------
        -- Active-low Reset
        ---------------------------------------------------------------------
        IF reset = '0' THEN
            Result <= (others => '0');
            neg    <= '0';

        ELSIF rising_edge(clock) THEN

            CASE opcode IS

                ---------------------------------------------------------------------
                -- 1. ADD : sum(A, B)
                ---------------------------------------------------------------------
                WHEN "0000000000000001" =>
                    neg    <= '0';
                    Result <= Reg1 + Reg2;

                ---------------------------------------------------------------------
                -- 2. SUB : diff(A, B) with neg flag
                ---------------------------------------------------------------------
                WHEN "0000000000000010" =>
                    IF Reg1 >= Reg2 THEN
                        neg    <= '0';
                        Result <= Reg1 - Reg2;
                    ELSE
                        neg    <= '1';
                        Result <= Reg2 - Reg1;
                    END IF;

                ---------------------------------------------------------------------
                -- 3. INVERSE : NOT(A)
                ---------------------------------------------------------------------
                WHEN "0000000000000100" =>
                    neg    <= '0';
                    Result <= NOT Reg1;
```

```
                -------------------------------------------------------------
                -- 3. INVERSE : NOT(A)
                -------------------------------------------------------------
                WHEN "0000000000000100" =>
                    neg    <= '0';
                    Result <= NOT Reg1;

                -------------------------------------------------------------
                -- 4. NAND : NOT(A AND B)
                -------------------------------------------------------------
                WHEN "0000000000001000" =>
                    neg    <= '0';
                    Result <= NOT (Reg1 AND Reg2);

                -------------------------------------------------------------
                -- 5. NOR : NOT(A OR B)
                -------------------------------------------------------------
                WHEN "0000000000010000" =>
                    neg    <= '0';
                    Result <= NOT (Reg1 OR Reg2);

                -------------------------------------------------------------
                -- 6. AND : A AND B
                -------------------------------------------------------------
                WHEN "0000000000100000" =>
                    neg    <= '0';
                    Result <= Reg1 AND Reg2;

                -------------------------------------------------------------
                -- 7. XOR : A XOR B
                -------------------------------------------------------------
                WHEN "0000000001000000" =>
                    neg    <= '0';
                    Result <= Reg1 XOR Reg2;

                -------------------------------------------------------------
                -- 8. OR : A OR B
                -------------------------------------------------------------
                WHEN "0000000010000000" =>
                    neg    <= '0';
                    Result <= Reg1 OR Reg2;

                -------------------------------------------------------------
                -- 9. XNOR : NOT(A XOR B)
                -------------------------------------------------------------
                WHEN "0000000100000000" =>
                    neg    <= '0';
                    Result <= NOT (Reg1 XOR Reg2);

                -------------------------------------------------------------
                -- DEFAULT CASE: No Operation
                -------------------------------------------------------------
                WHEN OTHERS =>
                    Result <= Result;

            END CASE;

        END IF;
    END PROCESS;

    -------------------------------------------------------------------------
    -- Split the 8-bit result into two 4-bit outputs for 7 segment display
    -------------------------------------------------------------------------
    R1 <= Result(3 DOWNTO 0);       -- lower 4 bits
    R2 <= Result(7 DOWNTO 4);       -- upper 4 bits

END calculation;
```

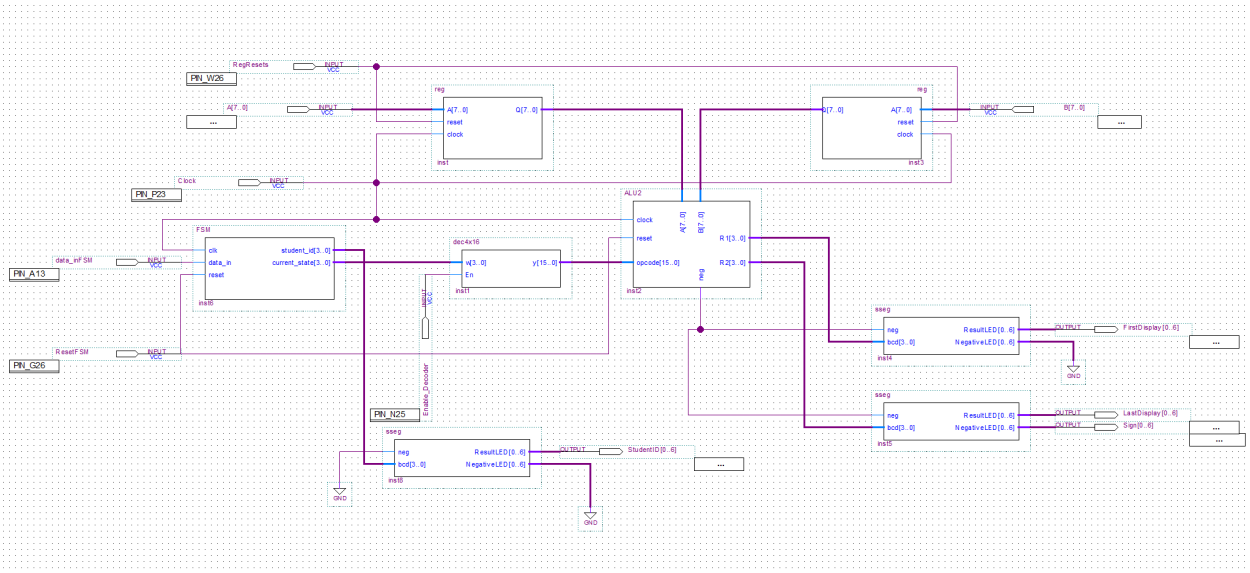
- Example Board Results:
  - 
  - 
  - 

5. ALU_2 for Problem Set 2 of the Lab6 procedure

- Description and Purpose: ALU_2 is an upgraded arithmetic logic unit that integrates nine special operations as defined in option C. This will demonstrate the ALU architecture's flexibility and programmability by substituting unique computational workloads for conventional operations. Efficiency in hardware has been kept within the architecture, while enabling a range of functions at arithmetic levels, bit-level manipulations, and data transformations.

- Design Approach: ALU_2 extends the base ALU_1 architecture by preserving the same interface while altering the underlying logic that executes the operations. This includes more complicated operations such as bit reversal, bit swapping, and conditional shifting that exploit both combinatorial and sequential logic. Each of these operations is accomplished in a single cycle to not affect uniform pipeline time.

- Block Schematic (BDF):



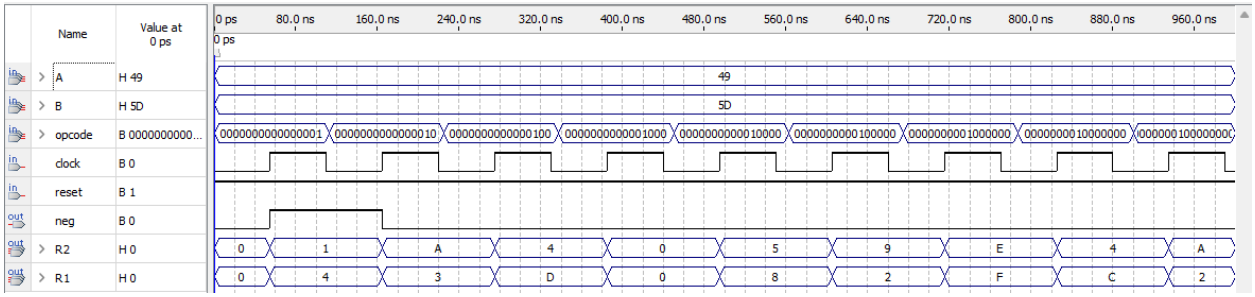
- Inputs and Outputs:
  - A and B (Input, 8-bit): Provides the operand used in all arithmetic and logic operations in ALU2.
  - Opcode (Input, 16-bit): Microcode from decoder, selects which specific ALU function to execute.
  - Clock (Input): System clock (Rising Edge), Output is updated on each rising edge.
  - Reset (Input): Active-low signal that clears the ALU result and resets the state.

- neg (Output): Indicates whether a subtraction operation produced a negative result (B > A).
- R1 (Output, 4-bit): Outputs the lower bits (3-0) of the 8-bit ALU result for the 7-segment display.
- R2 (Output, 4-bit): Outputs the upper bits (7-4) of the 8-bit ALU result for the 7-segment display.

- Table of Microcode's generated by decoder for ALU_2:
- Student ID: 501177393 -> A = 73 = 01001001 (0x49), B = 93 = 01011101 (0x5D)

Function #	Microcode (OP15 … OP0)	ALU2 Operation/Function	Results (Hex)
1	0000 0000 0000 0001	Difference between A and B	-14
2	0000 0000 0000 0010	2's complement of B	A3
3	0000 0000 0000 0100	Swap lower 4 bits of A with lower 4 bits of B	4D
4	0000 0000 0000 1000	Null output	00
5	0000 0000 0001 0000	Decrement B by 5	58
6	0000 0000 0010 0000	Bit-Reverse A (A0..A7)	92
7	0000 0000 0100 0000	Shift B left by 3, fill with "111"	EF
8	0000 0000 1000 0000	Increment A by 3	4C
9	0000 0001 0000 0000	Invert all bits of B	A2

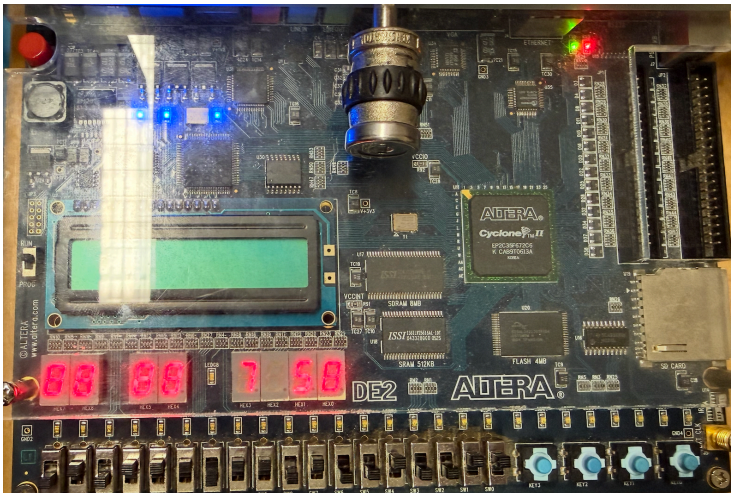
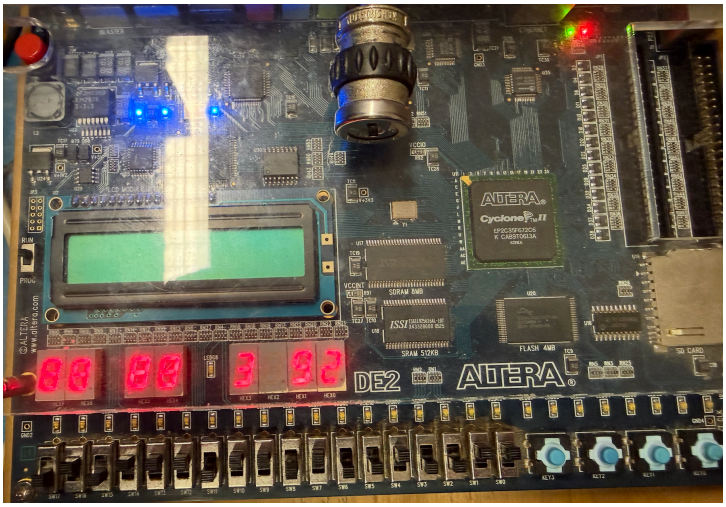
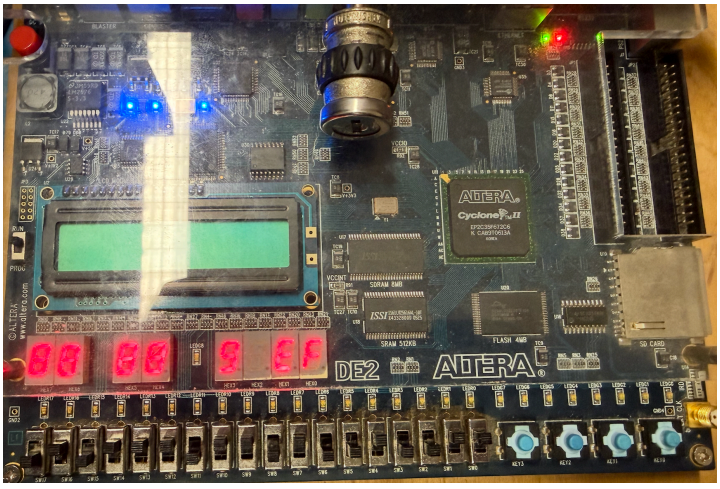
- Complete Waveform File:



- VHDL Code:

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;
USE ieee.std_logic_unsigned.all;
USE ieee.numeric_std.all;

ENTITY ALU2 IS
    PORT (
        clock  : IN  std_logic;
        reset  : IN  std_logic;              -- active-low reset
        A, B   : IN  unsigned(7 downto 0);   -- 8-bit inputs
        opcode : IN  unsigned(15 downto 0);  -- 16-bit opcode from decoder
        neg    : OUT std_logic;              -- negative flag
        R1, R2 : OUT unsigned(3 downto 0)    -- 4-bit outputs for 7-seg
    );
END ALU2;

ARCHITECTURE calculation OF ALU2 IS

    SIGNAL Reg1, Reg2, Result : unsigned(7 downto 0) := (others => '0');

BEGIN
    ---------------------------------------------------------------------
    -- Store inputs in temporary registers
    ---------------------------------------------------------------------
    Reg1 <= A;
    Reg2 <= B;

    PROCESS(clock, reset)
    BEGIN
        ---------------------------------------------------------------------
        -- Active-low Reset
        ---------------------------------------------------------------------
        IF reset = '0' THEN
            Result <= (others => '0');
            neg    <= '0';

        ELSIF rising_edge(clock) THEN

            CASE opcode IS

                ---------------------------------------------------------------------
                -- 1. Produce the difference between A and B
                --    (set neg = '1' if result is "negative" i.e. B > A)
                ---------------------------------------------------------------------
                WHEN "0000000000000001" =>
                    IF Reg1 >= Reg2 THEN
                        neg    <= '0';
                        Result <= Reg1 - Reg2;     -- A - B
                    ELSE
                        neg    <= '1';
                        Result <= Reg2 - Reg1;     -- |A - B|
                    END IF;

                ---------------------------------------------------------------------
                -- 2. Produce the 2's complement of B
                ---------------------------------------------------------------------
                WHEN "0000000000000010" =>
                    neg    <= '0';
                    Result <= (NOT Reg2) + 1;

                ---------------------------------------------------------------------
                -- 3. Swap the lower 4 bits of A with lower 4 bits of B
                --    Output = A with its low bits replaced by B's low bits
                --    [A7..A4][B3..B0]
                ---------------------------------------------------------------------
                WHEN "0000000000000100" =>
                    neg    <= '0';
                    Result <= Reg1(7 downto 4) & Reg2(3 downto 0);
```

```
                WHEN "0000000000000100" =>
                    neg    <= '0';
                    Result <= Reg1(7 downto 4) & Reg2(3 downto 0);

                ------------------------------------------------------------
                -- 4. Produce null on the output (all zeros)
                ------------------------------------------------------------
                WHEN "0000000000001000" =>
                    neg    <= '0';
                    Result <= (others => '0');

                ------------------------------------------------------------
                -- 5. Decrement B by 5
                ------------------------------------------------------------
                WHEN "0000000000010000" =>
                    neg    <= '0';
                    Result <= Reg2 - 5;

                ------------------------------------------------------------
                -- 6. Invert the bit-significance order of A
                --    Bit-reverse: [A7..A0] -> [A0..A7]
                ------------------------------------------------------------
                WHEN "0000000000100000" =>
                    neg    <= '0';
                    Result <= Reg1(0) & Reg1(1) & Reg1(2) & Reg1(3) &
                              Reg1(4) & Reg1(5) & Reg1(6) & Reg1(7);

                ------------------------------------------------------------
                -- 7. Shift B left by three bits, input bit = 1 (SHL)
                --    Result = B << 3 with "111" filled on the right
                --    [B4..B0][1 1 1]
                ------------------------------------------------------------
                WHEN "0000000001000000" =>
                    neg    <= '0';
                    Result <= Reg2(4 downto 0) & "111";

                ------------------------------------------------------------
                -- 8. Increment A by 3
                ------------------------------------------------------------
                WHEN "0000000010000000" =>
                    neg    <= '0';
                    Result <= Reg1 + 3;

                ------------------------------------------------------------
                -- 9. Invert all bits of B
                ------------------------------------------------------------
                WHEN "0000000100000000" =>
                    neg    <= '0';
                    Result <= NOT Reg2;

                ------------------------------------------------------------
                -- DEFAULT CASE: No Operation
                ------------------------------------------------------------
                WHEN OTHERS =>
                    Result <= Result;
            END CASE;

        END IF;
    END PROCESS;

    ------------------------------------------------------------------------
    -- Split the 8-bit result into two 4-bit outputs for 7 segment display
    ------------------------------------------------------------------------
    R1 <= Result(3 DOWNTO 0);     -- lower 4 bits
    R2 <= Result(7 DOWNTO 4);     -- upper 4 bits

END calculation;
```

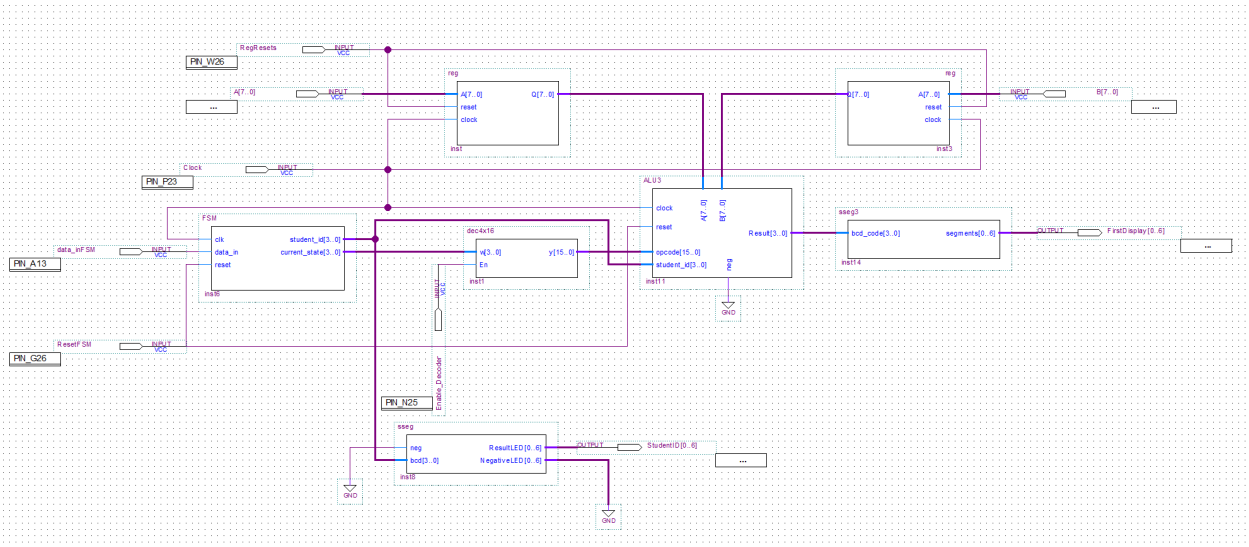
- Example Board Results:
  - 
  - 
  - 

6. ALU_3 for Problem Set 3 of the Lab6 procedure

- Description and Purpose:

  ALU_3 implements a conditional evaluation unit which checks the parity of the student_id from the FSM. The goal is to demonstrate how conditional processing in a processor pipeline can be driven by information from a control unit. Instead of calculating data, ALU_3 assesses a control state property and outputs a boolean result for display.

- Design Approach: The number of "1s" in the 4-bit student_id is counted by the parity detection logic. An XOR reduction circuit detects odd parity by determining whether the XOR of all bits equals 1. The outcome is encoded onto a 4-bit output pattern that powers a specially designed 7-segment display set up to show the characters "y" or "n."
- Block Schematic (BDF):

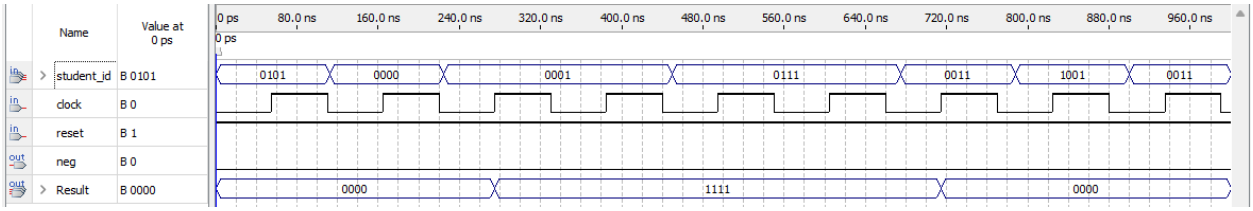


- Inputs and Outputs:
  - A and B (Input, 8-bit): Operand kept for ALU1/ALU2, not used in ALU3.
  - Opcode (Input, 16-bit): Microcode from decoder, but not used in ALU3.
  - Student_id (Input, 4-bit): Output from FSM.
  - Clock (Input): System clock (Rising Edge), Output is updated on each rising edge.
  - Reset (Input): Active-low signal that clears the Result to “0000”.
  - Result (Output, 4-bit): Parity result from ALU3, “1111” = 'y' when student_id has odd parity, “0000” = 'n' otherwise.
  - neg (Output): Indicates a negative result, not used in ALU3.

- Truth Table for ALU_3 (Opcode, A and B not used in ALU_3 Problem):

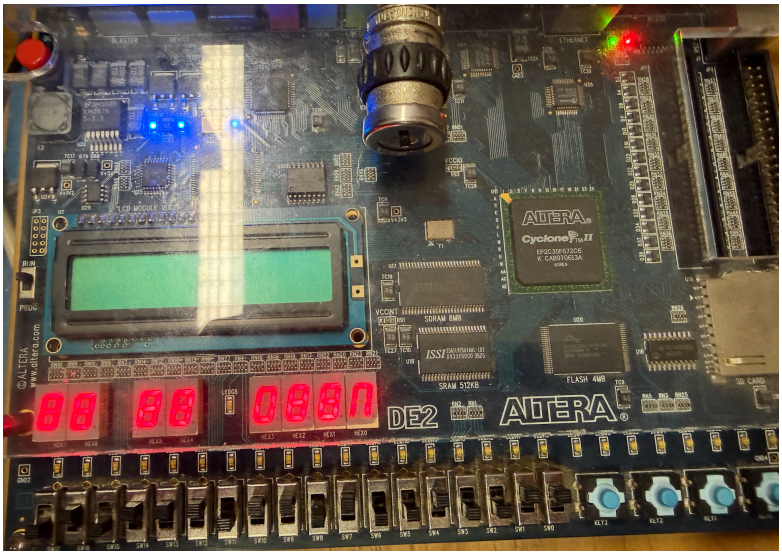
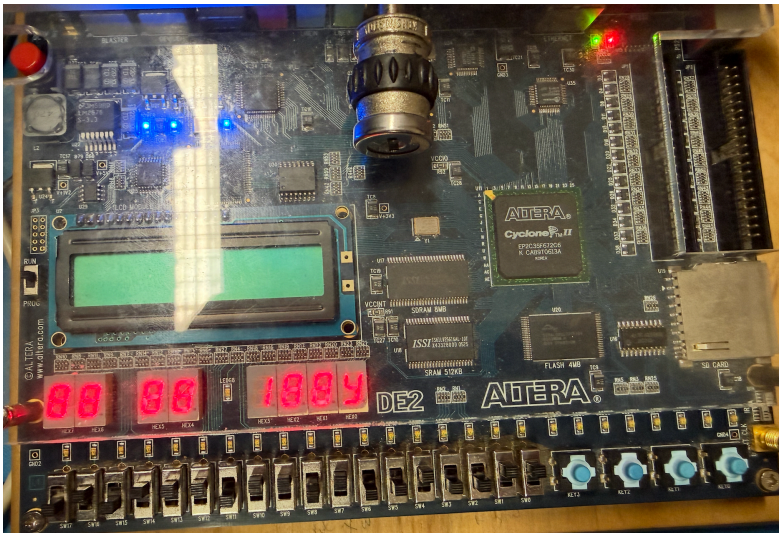
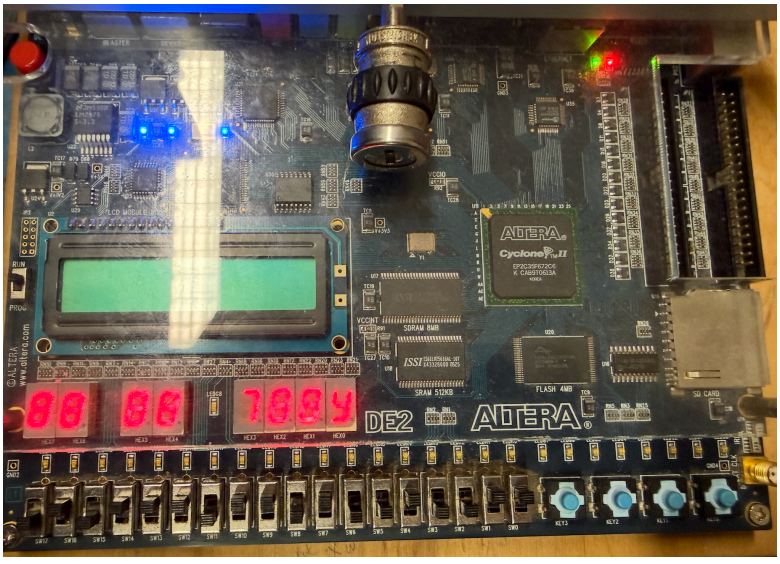
FSM	Student Digit	Student_id (Binary)	# of 1's	Parity	ALU3 (Result)	7-Segment Display Output
S0	5	0101	2	Even	0000	n
S1	0	0000	0	Even	0000	n
S2	1	0001	1	Odd	1111	y
S3	1	0001	1	Odd	1111	y
S4	7	0111	3	Odd	1111	y
S5	7	0111	3	Odd	1111	y
S6	3	0011	2	Even	0000	n
S7	9	1001	2	Even	0000	n
S8	3	0011	2	Even	0000	n

- Complete Waveform File:



- VHDL Code:

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;
USE ieee.std_logic_unsigned.all;
USE ieee.numeric_std.all;

ENTITY ALU3 IS
    PORT (
        clock      : IN  std_logic;
        reset      : IN  std_logic;                     -- active-low reset
        A, B       : IN  unsigned(7 downto 0);          -- 8-bit inputs
        opcode     : IN  unsigned(15 downto 0);         -- 16-bit opcode from decoder
        student_id : IN  std_logic_vector(3 downto 0); -- FSM output
        Result     : OUT std_logic_vector(3 downto 0); -- 1111 = 'y', 0000 = 'n'
        neg        : OUT std_logic                      -- not used
    );
END ALU3;

ARCHITECTURE calculation OF ALU3 IS

    -- 1 = student_id has odd number of "1" bits
    SIGNAL odd_parity : std_logic;

BEGIN
    ------------------------------------------------------------------------------------------------------------------------
    -- 3. C) For each microcode instruction, display 'y' if the FSM output (student_id) had an odd parity and 'n' otherwise
    ------------------------------------------------------------------------------------------------------------------------
    odd_parity <= student_id(3) xor student_id(2) xor student_id(1) xor student_id(0);

    ----------------------------------------------------------------
    -- Output 'y' (1111) for odd parity or 'n' (0000)
    ----------------------------------------------------------------
    PROCESS(clock, reset)
    BEGIN
        IF reset = '0' THEN              -- default = 'n'
            Result <= "0000";
        ELSE                             --rising_edge(clock) THEN
            IF odd_parity = '1' THEN
                Result <= "1111";        -- means "y"
            ELSE
                Result <= "0000";        -- means "n"
            END IF;
        END IF;
    END PROCESS;

END calculation;
```

- Example Board Results:
  - 
  - 
  - 

7. Conclusion

The lab successfully created a basic processor that combined display units, data storage, ALU processing, and FSM control. ALU_1 offered the fundamental set of operations, while ALU_2 implemented user-defined operations, including specialized ones. ALU_3 displayed conditional processing, dependent on the outputs of the control unit. This project improved knowledge of digital system architecture, synchronous design, and computational system component integration.